Lab 1

1. The results of the first and last elements to confirm correct addition
2. Wrong output, because GPU cannot access the array we have in CPU RAM
3. Before kernel execution: a and b to GPU, after kernel execution: out from GPU back to CPU, pass to vector_add: GPU pointers
4. Same result as CPU version
5. Limits for x and y in Lab 1.1 = [-1,1], z = [0,1], if exceeded clipped and therby not visible on screen, why? Because only relevant stuff is rendered
6. Just use the VBO, but we can not reuse indices so there will be duplicated vertices
    - Model_mat4x4((1.000000, 0.000000, 0.000000, 0.000000), (0.000000, 1.000000, 0.000000, 0.000000), (0.000000, 0.000000, 1.000000, 0.000000), (0.000000, 0.000000, 0.000000, 1.000000))
    - View_mat4x4((1.000000, 0.000000, 0.000000, 0.000000), (0.000000, 1.000000, 0.000000, 0.000000), (0.000000, 0.000000, 1.000000, 0.000000), (0.000000, 0.000000, -5.000000, 1.000000))
    - Projection_mat4x4((1.810660, 0.000000, 0.000000, 0.000000), (0.000000, 2.414213, 0.000000, 0.000000), (0.000000, 0.000000, -1.020202, -1.000000), (0.000000, 0.000000, -2.020202, 0.000000))
7. The diagonale 1 scale the object (not if they are 1) in x,y,z and the last element [3,3] doesnt scale, but is set to 1 for transformation
8. [3,3] has the same porpose, the other 1 define the rotation, the -5 translates the scene
9. 1.8 [0,0] and 2.4 [1,1] scaling to objects, -1.02 [2,2] applies depth scaling and -1 [3][2] aswell as -2.02 [2,3] creates perspective distortion
10. GL_NEVER,GL_LESS,GL_EQUAL,GL_LEQUAL,GL_GREATER,GL_NOTEQUAL,GL_GEQUAL,GL_ALWAYS, yes it is appropriate, because it ensures that object that are closer to the camera will obscure objects farther away other options may not ensure this